

EMU环境PMU工具使用

1 概述

PMU (Performance Monitoring Unit) 是 SIPU 芯片内置的硬件性能计数器模块。每个硬件模块 (PE、DRAMIF、C2C 等) 都有一组计数器，可以在程序运行期间记录各种事件的发生次数或周期数，具体可参考架构设计文档：[SIPU 1.0 PFM\(Performance Monitor Unit\) Subsystem架构设计方案_v1.0](#)

PMU 计数器只做累加操作，硬件上没有为中间状态值预留存储空间，因此不支持分时采样 (time-sharing sampling)。软件对计数器只能执行三种操作：

1. **Reset** — 清零计数器
2. **Stop** — 停止计数
3. **Read** — 读取当前累计值

这意味着每次采集只能获得从 reset 到 stop 之间的累计总数，无法获取过程中的时间序列数据。

2 代码仓

主仓库：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool

版本发布地址：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/releases

说明文档：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/blob/dev/README.md?ref_type=heads

技术细节介绍：

https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/blob/dev/docs/pmu_tool_guide.md?ref_type=heads

3 基础使用方法



1. 为保证性能分析准确，请尽量使用带有buffer die的EMU环境
2. 需要根据实际运行的EMU环境，与kernel实际使用的PE，选择合适的mask profile进行指令执行

3.1 环境准备

PMU工具一般在真实设备上（或真实仿真环境，如ZEBU EMU）使用，如果在ZEBU上使用，需要首先配置启动EMU等环境，以下均以ZEBU EMU环境为例进行使用说明

3.2 版本获取

获取最新稳定版本的PMU发布版，形如：PMU Tool v20260325，可从上面列出的版本发布地址中获取。介质形式为压缩包，需要拷贝至EMU/QEMU环境中并解压

3.3 介质验证

确保QEMU已正常启动，且SIPU driver已加载，可尝试执行./pmu_tool --help观察是否可以正常输出内容

3.4 kernel准备

准备待测试分析的kernel binary，如tile_sum，确保binary具有可执行权限，且在当前路径下可正常运行

3.5 性能分析指令

使用指令：./pmu_tool ./**KERNEL_PATH**/tile_sum（注意，kernel binary前需要./前缀）
观察仿真输出结果是否正常，因为默认pmu工具会多次执行kernel来获取完整数据，kernel输出内容也会重复多次（确保当前kernel输出结果，与单独运行kernel时，一致）
如果需要使用其他更为细致的分析方式，可参考PMU工具的用法介绍文档。

3.6 结果查看

在当前目录下，会生成output目录，内部按时间戳生成已执行pmu工具分析的kernel执行结果，形如：output/pmu_20260326_110031/，包含文件一般有：

代码块

```
1  -rw-rw-r-- 1 ceshi ceshi 30295  3月 26 11:00 counters_organized.txt
2  -rw-rw-r-- 1 ceshi ceshi 45869  3月 26 11:00 counters.txt
3  -rw-rw-r-- 1 ceshi ceshi  1063  3月 26 11:00 hwconfig.txt
4  -rw-rw-r-- 1 ceshi ceshi 98669  3月 26 11:00 metrics_organized.txt
5  -rw-rw-r-- 1 ceshi ceshi 65556  3月 26 11:00 metrics.txt
6  -rw-rw-r-- 1 ceshi ceshi  3373  3月 26 11:00 rvcore_organized.txt
7  -rw-rw-r-- 1 ceshi ceshi  4385  3月 26 11:00 rvcore.txt
```

可主要关注其中的counters.txt文件，内部包含各个类型counter的信息，形如：

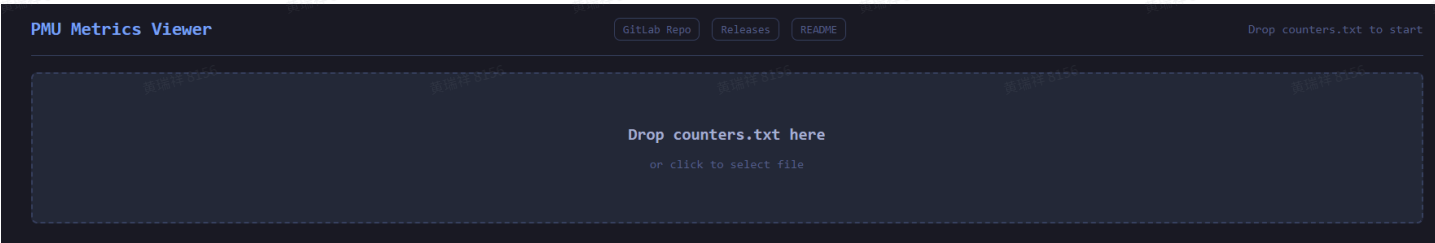
代码块

```
1  # Device: usipu0, Module: PE, PFM: 0, Group: 0
```

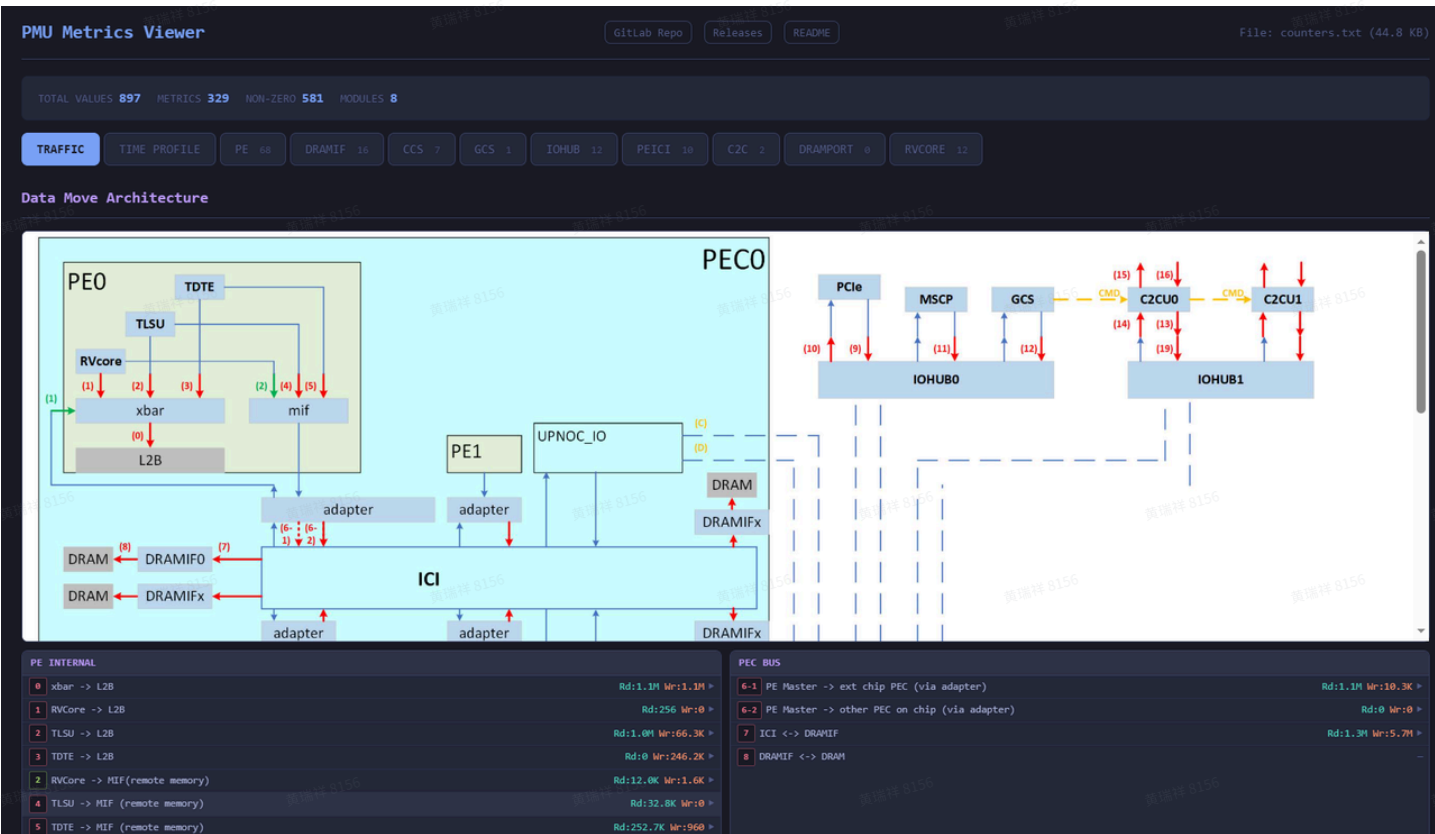
2	usipu0	PE	0	0	0	pe_active_cycles	0
3	usipu0	PE	0	0	1	rvcore0_active_cycles	0
4	usipu0	PE	0	0	2	rvcore1_active_cycles	0
5	usipu0	PE	0	0	3	tlsu_ld_active_cycle	0
6	usipu0	PE	0	0	4	tlsu_st_active_cycle	0
7	usipu0	PE	0	0	5	talu_active_cycles	0
8	usipu0	PE	0	0	6	tsfu_active_cycles	0
9	usipu0	PE	0	0	7	tdte_active_cycles	0
10	usipu0	PE	0	0	8	l2b_active_cycles	0
11	usipu0	PE	0	0	9	tmac_executing	0
12	usipu0	PE	0	0	10	pme_tsfu_fp16_inst_num	0
13	usipu0	PE	0	0	11	pme_tsfu_bf16_inst_num	0
14	usipu0	PE	0	0	12	pme_tsfu_fp32_inst_num	0

3.7 可视化工具

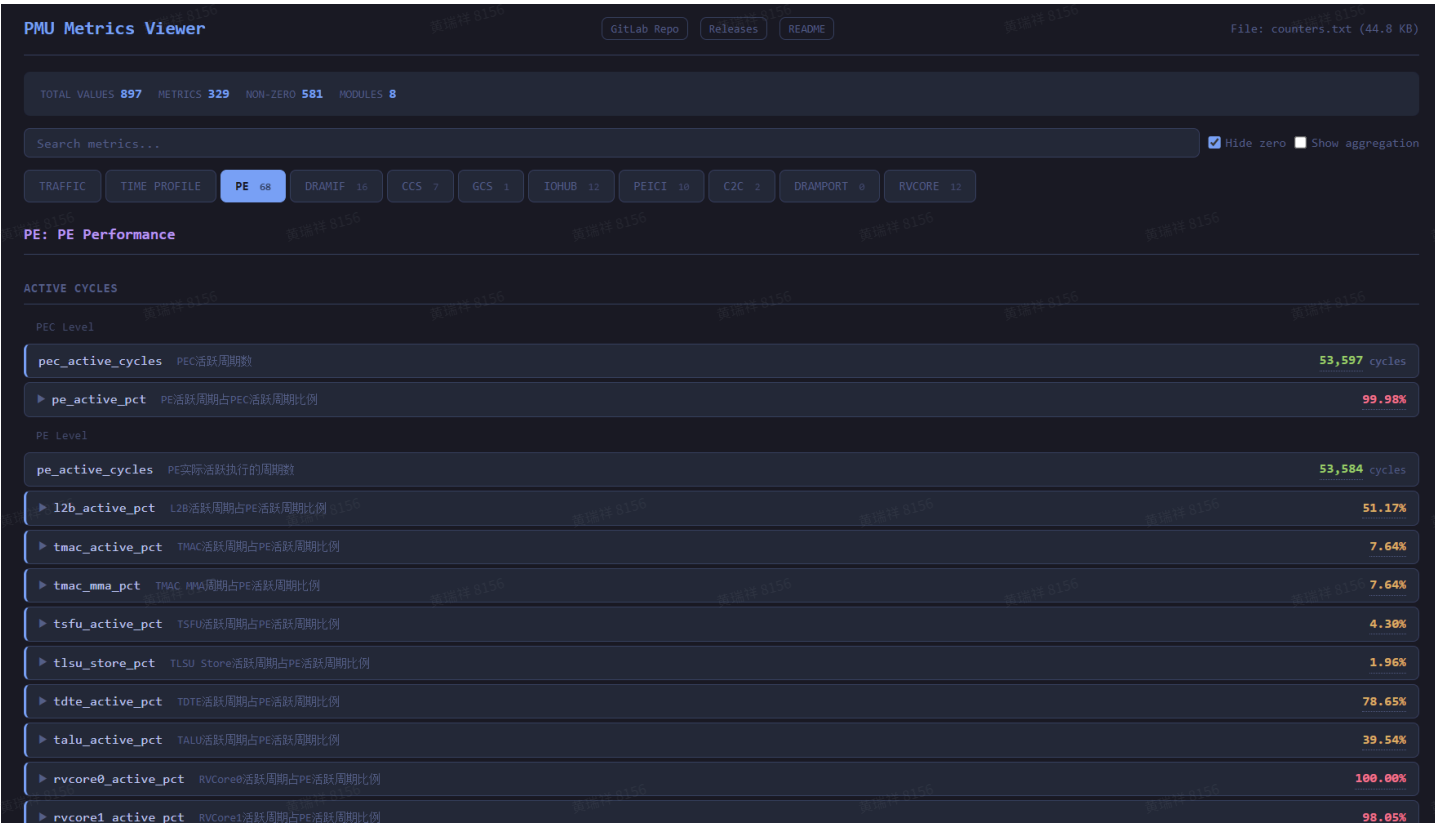
可使用可视化工具，查看counters.txt的结果，可视化工具与版本发布包在同一目录下，名为：metrics_viewer.html，可直接用浏览器打开该文件，有类似如下结果：



可用文件拖动的方式，将关注的counters.txt文件拖拽至其中，有类似如下输出：



点击PE栏目，可看到PE相关指令的执行统计情况：



也可点击其他栏目，查看关注的相关信息。

3.8 AI辅助分析

也可考虑将生成的counters.txt文件投喂给AI coding工具，用于辅助分析kernel的性能和优化方向。

4 可选的profile配置

需要注意的是，当前的EMU环境分为多个版本，不同版本能够抓取的信号，也是不同的，常用的EMU版本（仅针对sikernel的运行需要）包括：

- (1) **S1G1C1E**
最小形态，仅包含1个PE
 - (2) **S1G1C4E**
包含1个完整的PEC（内含4个PE）
 - (3) **S1G4C4E**
包含1个完整的PEG（内含4个完整的PEC）
- 另外还有更大的S2G4C4E，以及多卡版本，在sikernel的运行过程中，是不需要额外的这些硬件资源的，这里就不进行展开介绍了。

下面就针对上面这3种EMU版本，说明其profile的配置方法（是否带有buffer die，profile均相同；同时，sikernel不关注C2C，因此这里均不配置抓取）。

以下配置内容，需要补充到pmu工具配置文件（config/hardware_config.json）的尾部，为避免数值交叉，此处选取较大的profile id。

注意：

1. 非所有PEC或PE生效的情况下，需要结合实际EMU版本，查看active的PE或PEC的位置，可能与下面的配置有差异，比如S1G1C1E版本，可能最后1个PE为active，那么就需要把PE_MASK_PER_PEC改为[0, 0, 0, 1]
2. 不带有buffer die的版本，需要将DRAMPORT_MASK_PER_PEC参数置零："DRAMPORT_MASK_PER_PEC": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]，否则会导致EMU hang（需要重启EMU）

(1) S1G1C1E

代码块

```
1      "51": {
2          "_info": "S1G1C1E",
3          "PEC_MASK": [1, 0, 0, 0, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 0, 0, 0],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0],
8          "GCS_MASK": [1],
9          "IOHUB_MASK": [1, 1],
10         "IOPD_MASK": [1, 1],
11         "C2C_MASK": [0, 0]
12     }
```

(2) S1G1C4E

代码块

```
1      "52": {
2          "_info": "S1G1C4E buffer die",
3          "PEC_MASK": [1, 0, 0, 0, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 1, 1, 1],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1],
```

```

8      "GCS_MASK": [1],
9      "IOHUB_MASK": [1, 1],
10     "IOPD_MASK": [1, 1],
11     "C2C_MASK": [0, 0]
12 }

```

(3) S1G4C4E

代码块

```

1      "53": {
2          "_info": "S1G4C4E buffer die",
3          "PEC_MASK": [1, 1, 1, 1, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 1, 1, 1],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1],
8          "GCS_MASK": [1],
9          "IOHUB_MASK": [1, 1],
10         "IOPD_MASK": [1, 1],
11         "C2C_MASK": [0, 0]
12     }

```

根据实际的EMU版本，选择profile，执行PMU工具指令，如下：

代码块

```

1  # 选择S1G1C4E环境
2  cd /PMU_PATH
3  ./pmu_tool --mask-profile 52 /KERNEL_PATH/tile_sum

```